Analysis of Algorithms: Methodologies

Prof. Rohit P. Singh

Department of Computer Science

University of Wisconsin, Milwaukee

September 9, 2024

Readings: Chapter 1.1, 1.2

Some Definitions

- An algorithm is a step-by-step procedure for solving a problem in a finite amount of time

- A data structure is a systematic way of organizing and accessing data required in an algorithm

- Different algorithms for the same problem
  - *Example*: Finding the maximum of an array

- Analysis of algorithm: good vs. bad algorithms
  - **Running time**
  - Memory space
  - Robustness (stability)
  - Complexity
  - ....

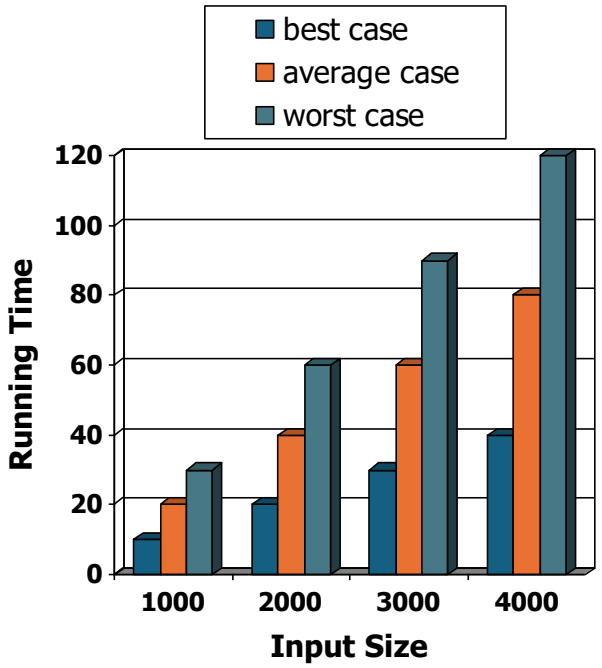
Running Time

- An algorithm transforms inputs into outputs.

- The running time of an algorithm typically grows with the input size. Even with the same input size, the running time can be different

- Our big goal is to design algorithms that scale well (or efficient algorithms)

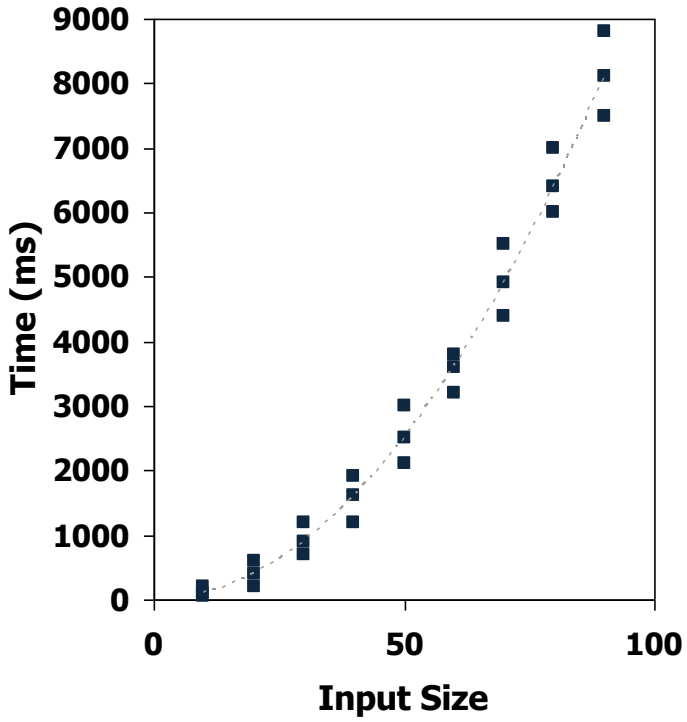
- We often consider the worst-case running time.
  - Easier to analyze
  - Crucial to some applications as games, finance and robotics



Estimation of Running Time

Experimental approach:

- Write a program implementing the algorithm
- Run the program with inputs of varying sizes
- Use a method to get an accurate measure of the actual running time
- Plot the results



Limitations of Experimental Approach

- It is necessary to implement the algorithm, which may be difficult and time consuming. If you have several algorithms to choose for a specific problem, you'll need to know the implementations of all of them.

- Even if you have all the implementations available, in order to compare two algorithms, the same hardware and software environments must be used.

- Experiments can be done only on a limited number of test inputs. Results may not accurately indicate the running time on other inputs not included in the experiment.

Theoretical Analysis

- Uses a high-level description of the algorithm instead of an implementation
- Takes into account all possible inputs
- Allows us to evaluate the speed of an algorithm independent of the hardware/software environment
- Characterizes running time as a function of the input size, such as $f(n)$.

Example 1: Algorithm A running in time proportional to n means that the running time of A on any input of size n is cn, where c a constant.

Example 2: If A runs in time proportional to n and B runs in time proportional to n^2, then A is preferred.

Pseudocode

- High-level description of an algorithm
- More structured than English words
- Less detailed and more readable than a program
- Preferred notation for describing algorithms
- Hides program design issues

Example of Pseudocode:

Find max element of an array

```
Algorithm arrayMax(A, n)
    Input array A of n integers
    Output maximum element of A

    currentMax ← A[0]
    for i ← 1 to n − 1 do
        if A[i] > currentMax then
            currentMax ← A[i]
    return currentMax
```

Pseudocode Details

- Method declaration

  Algorithm *method* (*arg* [, *arg*…])

  Input …

  Output …

- Control flow
  - if … then … [else …]
  - while … do …
  - repeat … until …
  - for … do …
  - Indentation replaces braces

- Method call

  object.method (***arg***s)

- Return value

  return ***expression***

- Expressions

  ← Assignment (like = in C)

  = Equality testing (like == in C)

  n^2 Superscripts and other mathematical formatting allowed

Primitive Operations

- Basic computations performed by an algorithm
- Identifiable in pseudocode
- Largely independent from the programming language or hardware environment

- Examples:
  - Evaluating an expression (e.g., addition, subtraction, etc.)
  - Assigning a value to a variable
  - Comparing two numbers
  - Indexing into an array
  - Calling a method
  - Returning from a method

The idea:

Count the number of primitive operations, to estimate the running time

Counting Primitive Operations: Iterative Algorithms

- By inspecting the pseudocode, we can determine the maximum number of primitive operations executed by an algorithm, as a function of the input size

Algorithm ***arrayMax***(A, n)	# operations
$currentMax \leftarrow A[0]$	2
for $i \leftarrow 1$ **to** $n - 1$ **do**	$n+1$ (if i < n?)
if $A[i] > currentMax$ **then**	$2(n - 1)$
$currentMax \leftarrow A[i]$	$2(n - 1)$ (Y/N?)
{ increment counter i }	$2(n - 1)$
return $currentMax$	1

at least: 5n (N)

at most: $7n - 2$ (Y)

Counting Primitive Operations: Recursive Algorithms

- By inspecting the pseudocode, we can determine the maximum number of primitive operations executed by an algorithm, as a function of the input size

	# operations
Algorithm ***recursiveMax***(***A***, ***n***)	
if n = 1 **then**	1
return A[0]	2
return ***max{recursiveMax(A,n-1),A[n-1]}***	6 + 1 (the **if** above)

base (n=1): 3

recursive: $T(n) = T(n-1) + 7$

Total: $T(n) = 7n - 4$

Estimating Running Time

- Algorithm $\boldsymbol{arrayMax}$ executes $7\boldsymbol{n} - 2$ primitive operations in the worst case. Define:
  - $\boldsymbol{a}$ = Actual time taken by the fastest primitive operation
  - $\boldsymbol{b}$ = Actual time taken by the slowest primitive operation

- Let $\boldsymbol{T}(\boldsymbol{n})$ be worst-case time of $\boldsymbol{arrayMax}$. Then

$$\boldsymbol{a}\,(7\boldsymbol{n} - 2) \leq \boldsymbol{T}(\boldsymbol{n}) \leq \boldsymbol{b}(7\boldsymbol{n} - 2)$$

- Hence, the running time $\boldsymbol{T}(\boldsymbol{n})$ is bounded by two linear functions

Growth Rate of Running Time

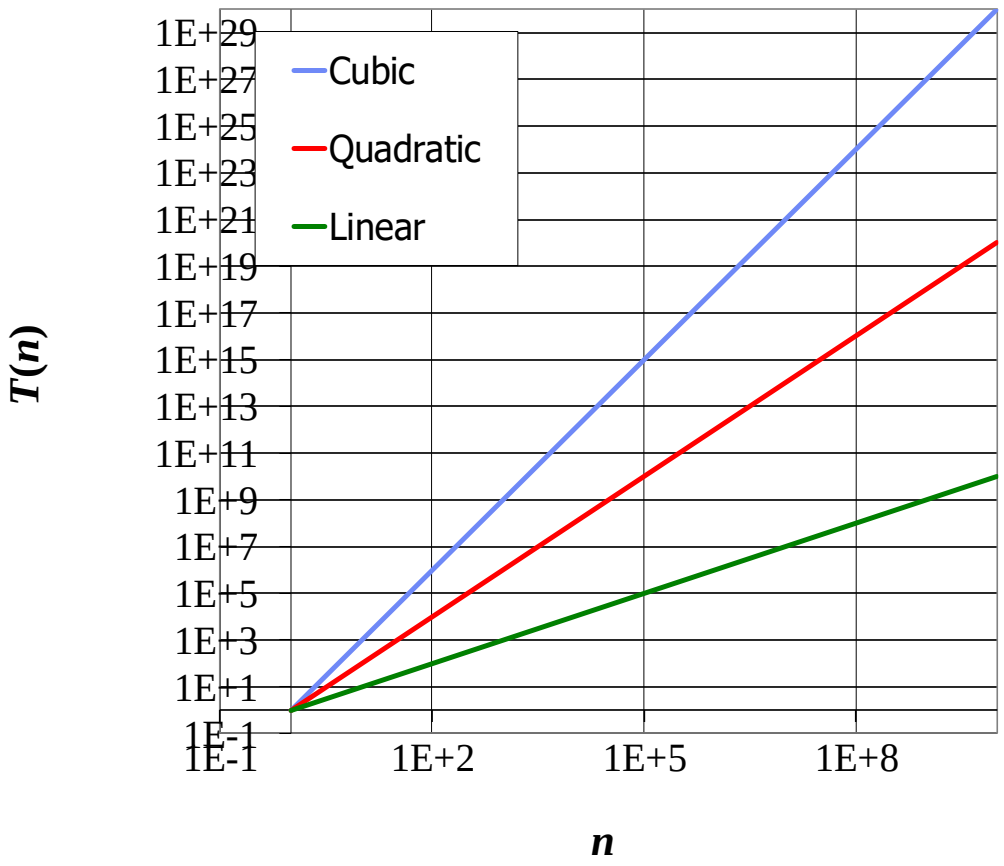
- Changing the hardware/software environments
  - Affects $\boldsymbol{T(n)}$ by a constant factor, but
  - Does not alter the growth rate of $\boldsymbol{T(n)}$

- The linear growth rate of the running time $\boldsymbol{T(n)}$ is an intrinsic property of algorithm $\boldsymbol{arrayMax}$

Growth Rates

- Growth rates of functions:
  - Linear $\approx \boldsymbol{n}$
  - Quadratic $\approx \boldsymbol{n}^2$
  - Cubic $\approx \boldsymbol{n}^3$

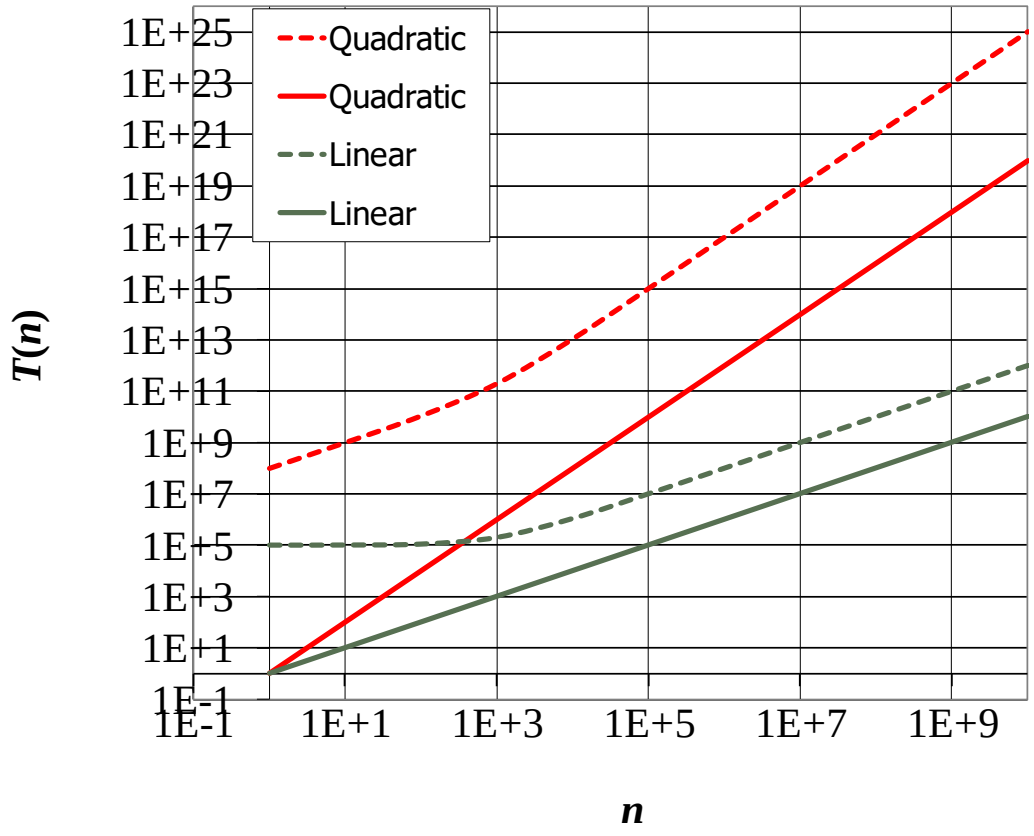
- In a log-log chart, the slope of the line corresponds to the growth rate of the function



Constant Factors

- The growth rate is not affected by
  - constant factors or
  - lower-order terms

- Examples
  - 102***n*** **+** 105 is a linear function
  - 105$\boldsymbol{n}^2$ **+** 108***n*** is a quadratic function

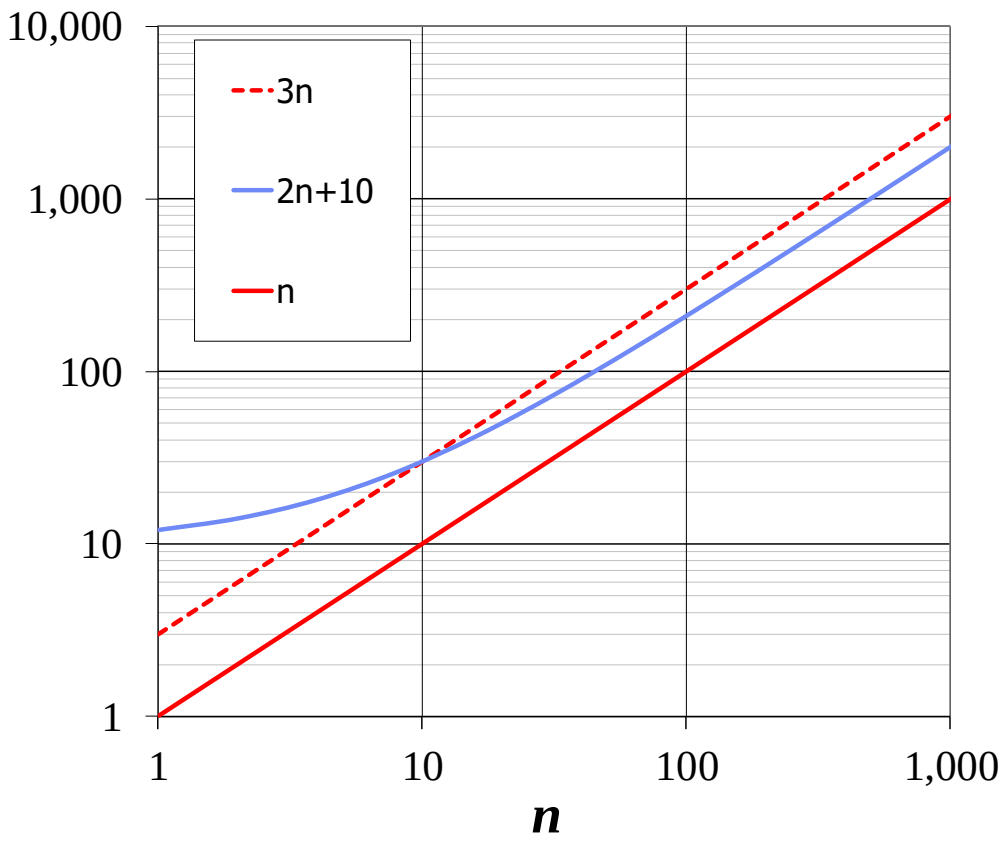


Big-Oh Notation

- Given functions $f(n)$ and $g(n)$, we say that $f(n)$ is $O(g(n))$ if there are positive constants c and n_0 such that:

  $f(n) \leq cg(n)$ for $n \geq n_0$

- Example: $2n + 10$ is $O(n)$
  - $2n + 10 \leq cn$
  - $(c - 2)\, n \geq 10$
  - $n \geq 10/(c - 2)$
  - Pick $c = 3$ and $n_0 = 10$
  - Or pick c = 4 and n_0 = 5

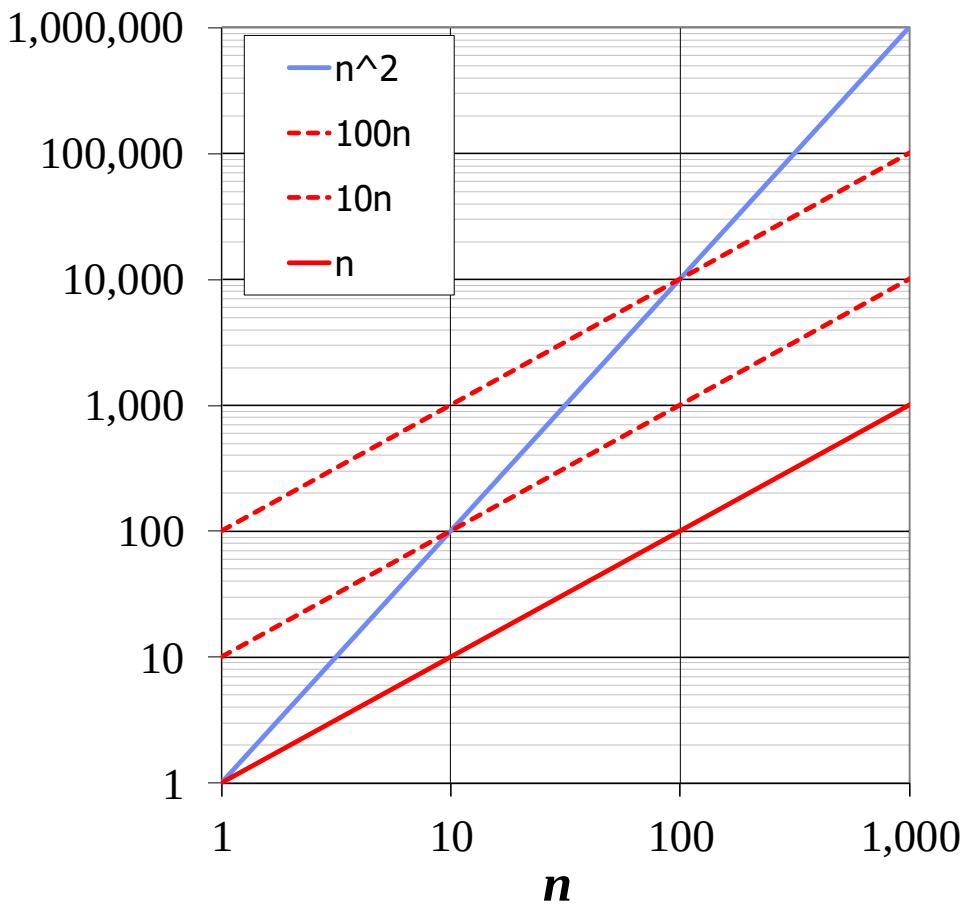


Big-Oh Example

- Example:

  the function n^2 is not $O(n)$

  - $n^2 \leq cn$
  - $n \leq c$
  - The above inequality cannot be satisfied since ***c*** must be a constant



Big-Oh Example (on your own)

- Show that 1000n is $O(2^n)$

n	1	2	3	4	5	6	7	8	9	10
1000n										
2^n										

But when n = 14, 1000n = 14,000 while 2^14 = 16,384.

So, when c = _____ and n0 = _____, we have:

$$1000n \leq c\ 2^n \text{ for } n \geq n_0$$

Big-Oh and Growth Rate

- The big-Oh notation gives an upper bound on the growth rate of a function
- The statement "$f(n)$ is $O(g(n))$" means that the growth rate of $f(n)$ is no more than the growth rate of $g(n)$
- We can use the big-Oh notation to rank functions according to their growth rate

	$f(n)$ is $O(g(n))$	$g(n)$ is $O(f(n))$
$g(n)$ grows more	Yes	No
$f(n)$ grows more	No	Yes
Same growth	Yes	Yes

Examples …

Big-Oh Rules

- In general, the following growth rate applies:

  O(1) O(log$n$) O(log^{2}n) O(sqrt(n)) O(n) O(nlogn) O(n^2) O(n^k) (k>=2) O(a^n) (a>1)

- If $\boldsymbol{f(n)}$ is a polynomial of degree $\boldsymbol{d}$, then $\boldsymbol{f(n)}$ is $\boldsymbol{O(n^d)}$, i.e.,
  1. Drop lower-order terms
  2. Drop constant factors

- Use the smallest possible class of functions
  - Say "$2\boldsymbol{n}$ is $\boldsymbol{O(n)}$" instead of "$2\boldsymbol{n}$ is $\boldsymbol{O(n^2)}$"

- Use the simplest expression of the class
  - Say "$3\boldsymbol{n} + 5$ is $\boldsymbol{O(n)}$" instead of "$3\boldsymbol{n} + 5$ is $\boldsymbol{O(3n)}$"

Some Other Useful Rules

- If $d(n)$ is $O(f(n))$, then $a*d(n)$ is $O(f(n))$ for any constant $a>0$.
- If $d(n)$ is $O(f(n))$ and $e(n)$ is $O(g(n))$, then $d(n)+e(n)$ is $O(f(n)+g(n))$.
- If $d(n)$ is $O(f(n))$ and $e(n)$ is $O(g(n))$, then $d(n)*e(n)$ is $O(f(n)*g(n))$.
- If $d(n)$ is $O(f(n))$ and $f(n)$ is $O(g(n))$, then $d(n)$ is $O(g(n))$